MentorDesk

PRODUCT REQUIREMENTS DOCUMENT

Intern Question & Mentor Response Portal

Document Control

Document field	Value
Version	1.0 — Planning draft
Organization	TriFin-Loop R&D
Project / owner	MentorDesk / Mahmuda Khatun
Status	Proposed for supervisor review; not approved for development
Frontend	React.js (proposed)
Backend	Node.js + Express.js (proposed; approval pending)
Database	PostgreSQL (proposed; approval pending)
Scope	A web application for interns to submit questions and mentors to respond and track status.
Document status	Planning document only; technical choices and open decisions require supervisor review.

This document describes a proposed MVP. It does not claim that the application has been implemented.

Executive Summary

MentorDesk is a proposed responsive web application that centralizes intern questions and mentor responses. Interns can submit questions, view their own question history, follow status changes, and add follow-up comments. Mentors can review submitted questions, provide responses, and update question status. The planned MVP emphasizes a clear question-to-response workflow, role-based access, and visible progress.

Version History

Version	Change
0.1	Initial problem, scope and requirements outline
1.0	Expanded planning PRD, diagrams, backlog, risks, API and test plans

01 | Problem, Users & Product Goals

Problem Statement

Intern questions and mentor replies may be spread across informal messages, email, or separate documents. This can make it difficult for interns to track progress and for mentors to review unresolved questions consistently. MentorDesk proposes a single place to submit questions, respond, update status, and review question history.

Target Users

Persona	Needs and primary actions
Intern	Submit a question with a title, description, category and priority; view own questions; follow status; add follow-up comments.
Mentor	View submitted questions; review details; add responses; update status; follow unresolved items.
Administrator (proposed)	Potentially manage users or roles and view operational summaries; exact responsibilities require supervisor approval.

Product Goals & Proposed Success Measures

- A valid intern question can be submitted once and appears in the intern's own question list.
- A mentor can review a question, add a response, and move it through the agreed status lifecycle.
- An intern cannot read or modify another intern's private questions by changing an identifier.
- Users can identify open and unresolved work from the dashboard or filtered question list.
- The primary workflow can be demonstrated using synthetic test accounts and sample questions.

Assumptions to Validate

- The project is a demonstration MVP and will use synthetic data only.
- Intern and Mentor are the primary roles; Administrator is proposed and needs confirmation.
- The proposed technology stack will be confirmed by the supervisor before development.
- Email notifications, file uploads, and production deployment are not assumed to be part of the initial MVP.

02 | Scope & MVP Feature Inventory

In Scope — Must Have

ID	Capability
F01	Register, sign in and sign out using role-aware accounts
F02	Create a question with title, category, description and priority
F03	View own questions and question details
F04	Mentor views submitted questions and reviews a question
F05	Mentor adds a response and updates question status
F06	Track status: Open, In Review, Answered, Resolved
F07	Add follow-up comments to a question
F08	Search/filter questions by status and relevant fields
F09	Dashboard summaries and unresolved-question view

F10	Server-side role and ownership checks

Should Have / Later

- Pagination for large question lists.
- Additional dashboard metrics and configurable categories.
- Email notifications after a response or status change, if approved.
- Accessibility and responsive refinements beyond the primary workflow.

Out of Scope for Initial Planning MVP

- AI-generated answers or automatic question classification.
- Live chat or video calls.
- Native mobile applications.
- Production SSO or integration with external identity providers.
- File uploads until storage, validation and access controls are designed.

Acceptance Boundary

The planned MVP is demonstrable when an intern can submit and track a question, a mentor can respond and update its status, and access restrictions prevent users from accessing records they do not own. A landing page alone does not meet this boundary.

03 | User Stories & Process Flow

Core User Stories

- US-01: As an intern, I want to submit a question so that a mentor can review it.
- US-02: As an intern, I want to view my question history and current status so that I know whether my question is being handled.
- US-03: As a mentor, I want to review submitted questions and filter unresolved work so that I can prioritize responses.
- US-04: As a mentor, I want to add a response and update status so that the intern can see progress.
- US-05: As an intern, I want to add a follow-up comment so that I can clarify a question or respond to a mentor.

Example Journey — Intern

1. The intern opens MentorDesk and signs in.
2. The intern opens the dashboard and selects Submit Question.
3. The intern enters title, category, description and priority.
4. The system validates required fields and saves the question with Open status.
5. The question appears in My Questions with a unique question identifier.
6. The intern opens the question detail page to review status, responses and follow-up comments.

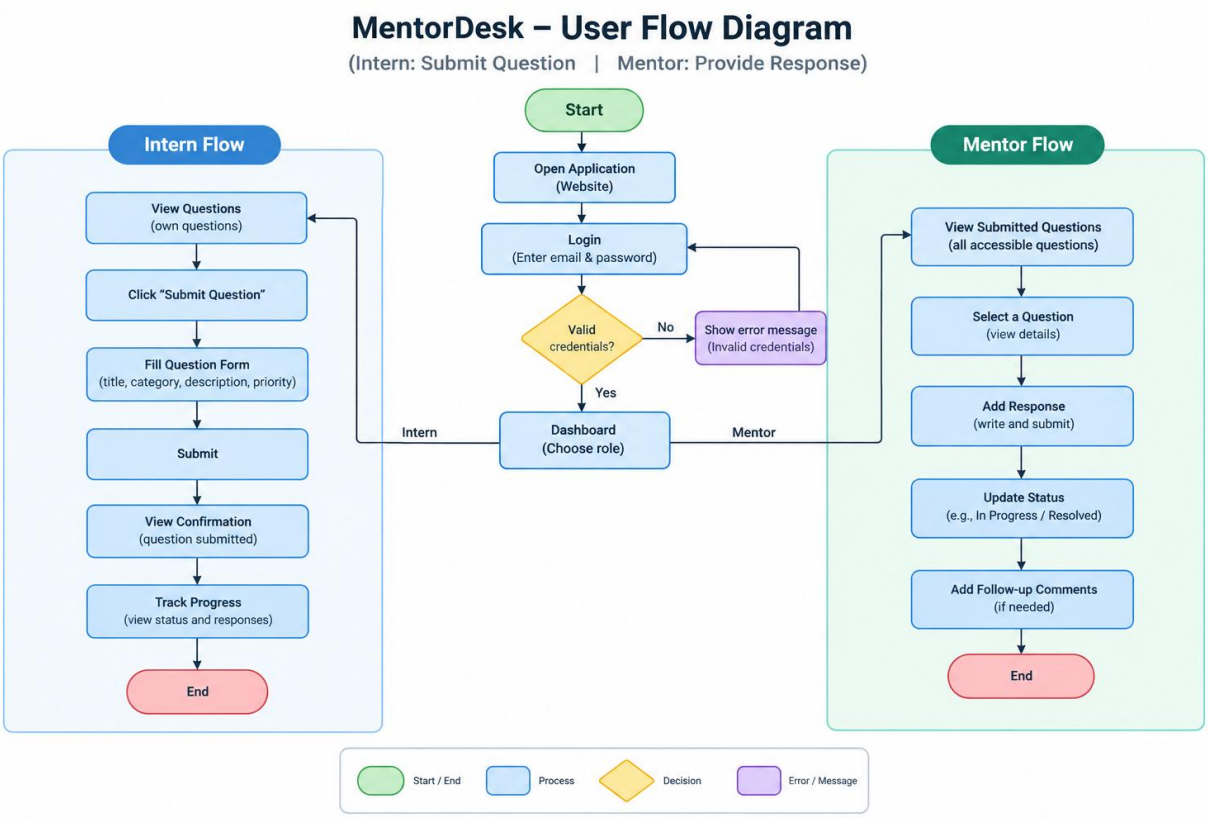
Example Journey — Mentor

7. The mentor signs in and opens the submitted-question list.
8. The mentor filters or selects a question and reviews its details.
9. The mentor adds a response and updates status to In Review or Answered as appropriate.
10. The intern can view the response and current status; follow-up comments may continue the conversation.
11. When the issue is complete, the question may be marked Resolved.

Proposed Lifecycle & Exceptions

Proposed status values: Open, In Review, Answered, and Resolved. The supervisor should confirm whether a Resolved question can be reopened. Invalid forms should remain unsaved and show clear field-level errors.

User Flow Diagram



04 | Functional Requirements

Requirement	Actor	Behavior	Acceptance criteria
FR-01 — Authentication	Intern/Mentor	User signs in or out using valid credentials; session state is handled safely.	Valid credentials open the appropriate dashboard; invalid credentials show a generic error.
FR-02 — Submit question	Intern	Submit title, category, description and priority.	A valid submission creates one question, defaults to Open, and appears in My Questions.
FR-03 — View own questions	Intern	List and open the intern's own questions.	Only questions owned by the signed-in intern are returned.
FR-04 — Review questions	Mentor	View and open submitted questions.	Mentor sees the question list and details permitted by role.
FR-05 — Respond to question	Mentor	Add a response to a question.	Response is linked to the question and shows author and timestamp.
FR-06 — Update status	Mentor	Change status to an allowed lifecycle value.	Invalid transitions are rejected; status changes are recorded.
FR-07 — Follow-up comments	Intern/Mentor	Add a comment to a question thread.	Comment is linked to the question and author and displayed in order.
FR-08 — Search/filter	Intern/Mentor	Filter by status and search relevant question text.	Results respect role and ownership rules; empty states are helpful.
FR-09 — Dashboard summaries	Intern/Mentor	View counts and unresolved items relevant to the role.	Counts reflect the available authorized records.
FR-10 — Authorization	All roles	Access protected pages and API endpoints.	Server verifies identity, role and record ownership for every protected operation.

Validation Rules (Proposed)

- Title is required and should be limited to a reasonable length (proposed: 5–100 characters).
- Description is required and should be limited to a reasonable length (proposed: 20–2,000 characters).
- Category and priority must be selected from allowed values.
- All inputs are validated on the server, even when the frontend also validates them.
- Responses, comments and status changes must be associated with an authenticated user and a valid question.

05 | Non-Functional Requirements

ID	Requirement	Verification approach
NFR-01 Security	Hash passwords; enforce authorization on the server; validate input; keep secrets out of source control.	Attempt invalid login, role tampering and unauthorized record access.
NFR-02 Performance	Proposed target: common list views respond within 2 seconds on a local demo dataset of up to 1,000 questions.	Measure browser/network timing with seeded test data.
NFR-03 Usability	Primary workflow should work on mobile and desktop; forms have labels, visible focus and clear error messages.	Review keyboard navigation and viewport widths.
NFR-04 Data integrity	Question, response and comment records maintain valid relationships; failed operations should not leave partial records.	Test validation failures and forced database errors.
NFR-05 Maintainability	Separate frontend, API routes/business logic and data access; document setup in README.	Review repository structure and setup instructions.
NFR-06 Reliability	Show useful error states when a request fails; do not display false success messages.	Test unavailable API and failed save operations.
NFR-07 Compatibility	Support current mainstream desktop browsers and responsive mobile layouts.	Manual cross-browser and responsive checks.
NFR-08 Privacy	Use synthetic demo data only; restrict access	Review sample data and perform ownership

	to authorized records.	tests.

Dependencies & Constraints

Local development is proposed to require Node.js, npm, Git/GitHub, and PostgreSQL. Hosting is optional and requires supervisor approval. Do not place real private user information, credentials, or confidential questions in the demonstration project.

es marked proposed must be reviewed with the supervisor.

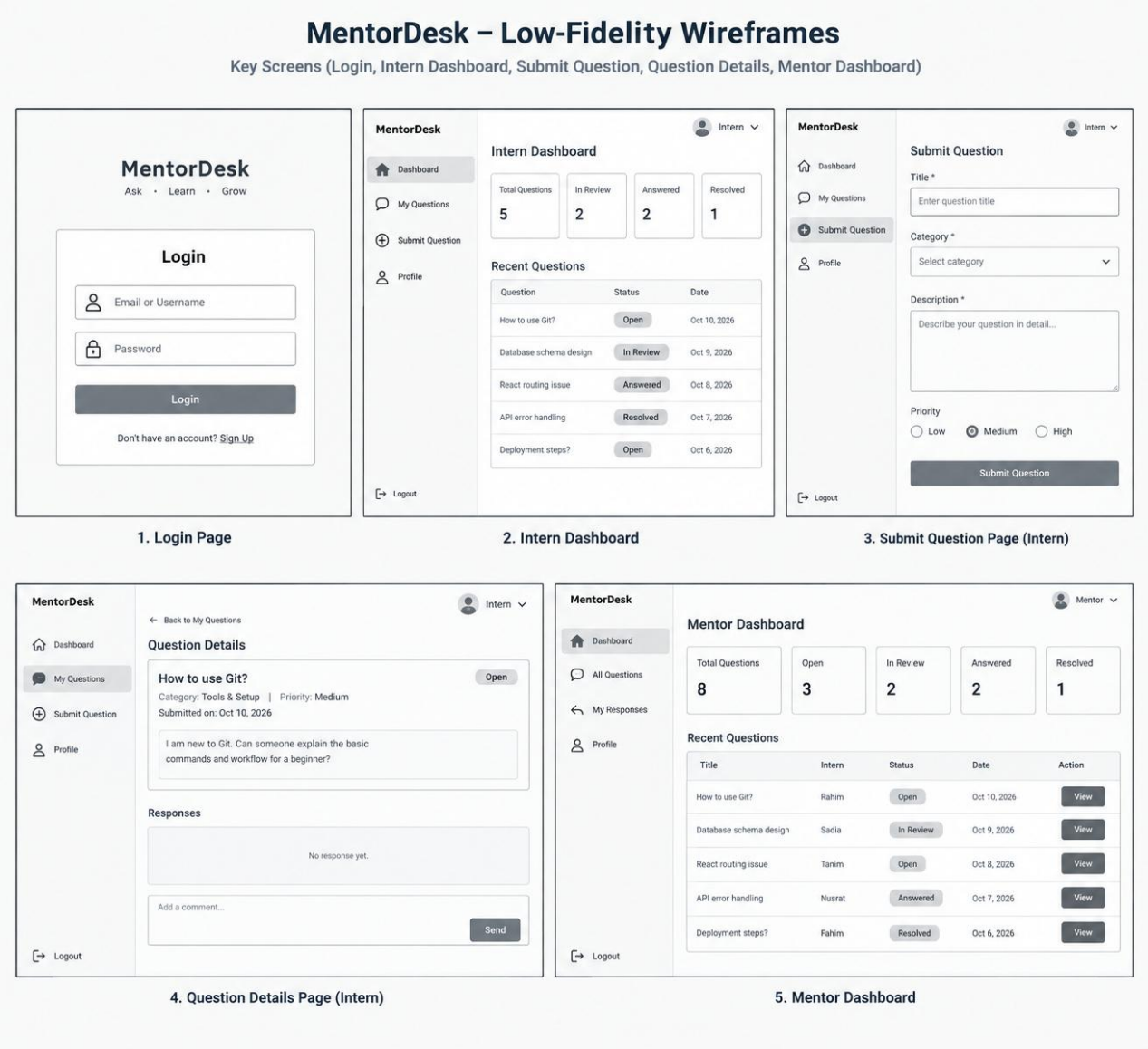
06 | UX, Screens & Low-Fidelity Wireframes

Screen	Required elements / behavior
Login	Email, password, validation feedback, sign-in action and role-aware redirect.
Intern Dashboard	Question summary, My Questions list, status labels, search/filter and Submit Question action.
Submit Question	Title, category, description, priority, submit/cancel controls, inline errors and success confirmation.
Question Details	Question ID, title, description, status, response thread, comments and last-updated time.
Mentor Dashboard	Submitted-question list, filters, question details, response field and status update controls.

Design Behavior

- Each screen identifies its primary user role, main action and navigation destination.
- Loading, empty, validation-error and success states should be planned for the main workflow.
- Interns must not see mentor-only response or status-management controls.
- A successful question submission should show confirmation and the question identifier.
- The main form should be considered at both mobile and desktop widths.

Low-Fidelity Wireframes

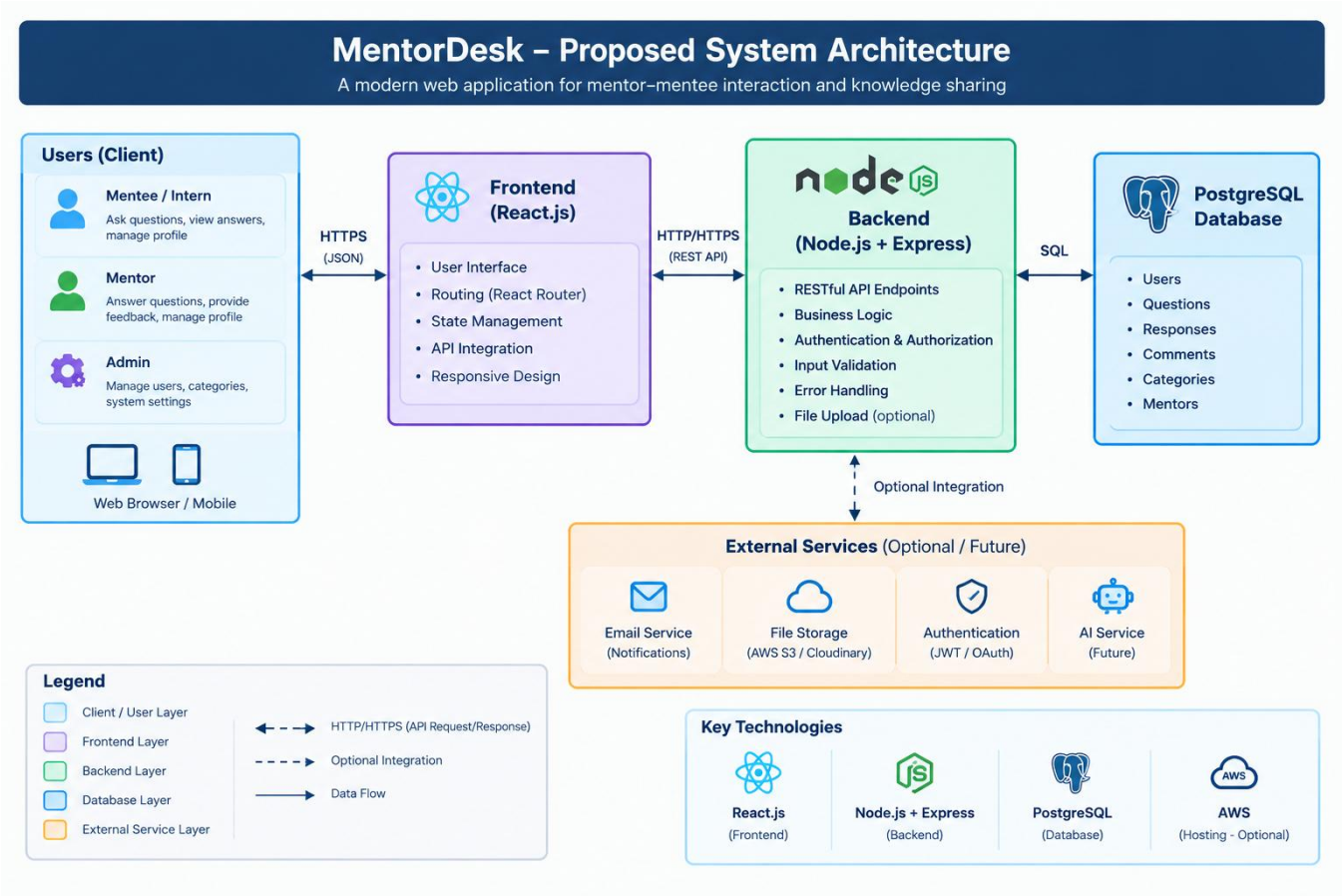


These wireframes are early layout sketches, not final visual designs. Confirm screen content and role permissions during supervisor review.

07 | Architecture, Data & API

Proposed Architecture

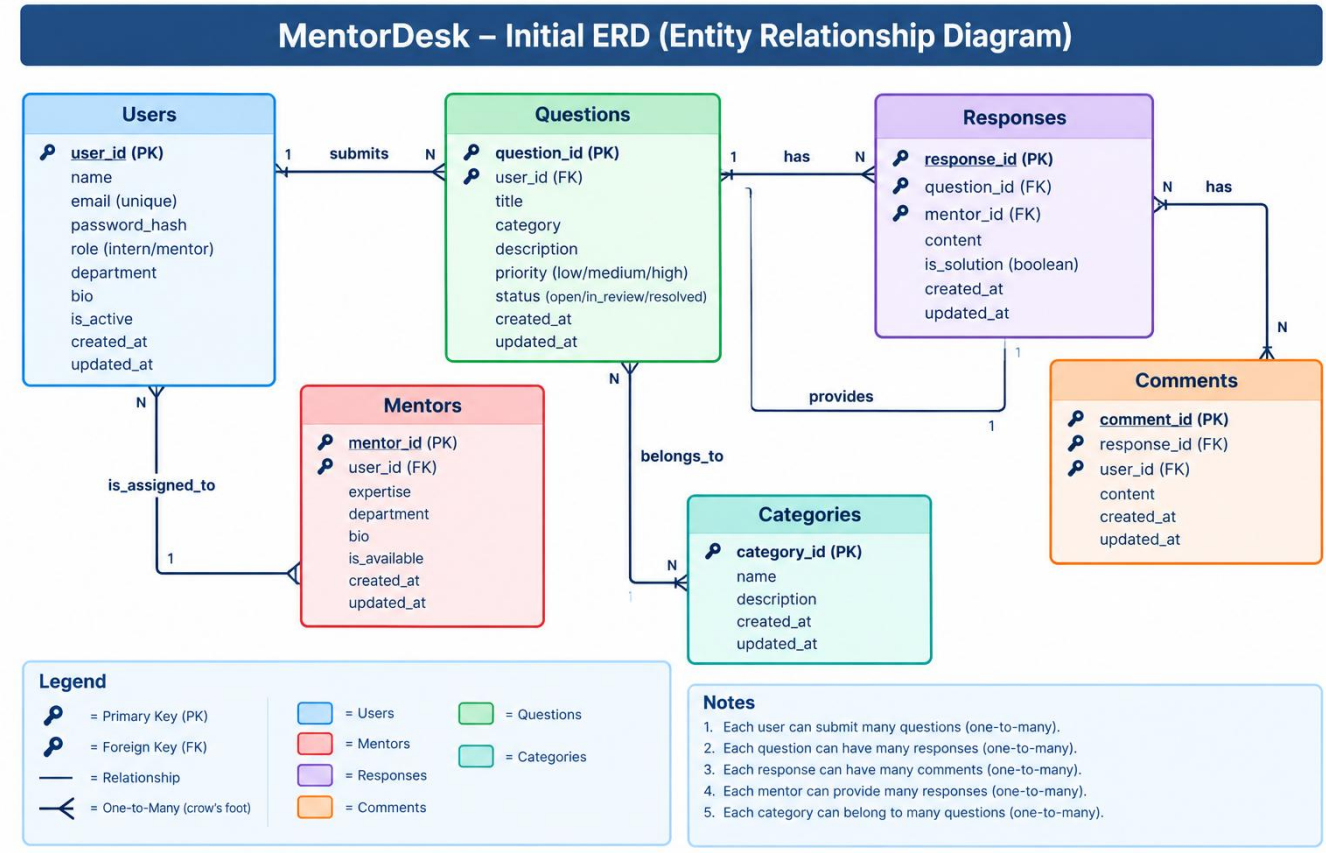
The React.js frontend communicates with a Node.js/Express.js REST API using HTTP(S) and JSON. The API handles authentication, authorization, validation and business logic, then reads/writes data in PostgreSQL. Optional email or file-storage integrations are not required for the initial MVP and must be separately approved.



Core Data Entities

Entity	Proposed fields	Purpose
Users	user_id (PK), name, email (unique), password_hash, role, created_at	Stores intern and mentor accounts.
Questions	question_id (PK), intern_id (FK), title, description, category, priority, status, created_at, updated_at	Stores questions submitted by interns.
Responses	response_id (PK), question_id (FK), mentor_id (FK), response_text, created_at	Stores mentor responses linked to questions.
Comments	comment_id (PK), question_id (FK), author_id (FK), comment_text, created_at	Stores follow-up comments by authorized participants.

Initial ERD



Relationship summary: one intern can submit many questions; one question can have many responses and comments; each response is written by one mentor; each comment is written by one user. Exact constraints and whether multiple mentors may respond to the same question should be confirmed.

Proposed API Endpoints

Method & endpoint	Purpose / access
POST /api/auth/register	Register an account, if self-registration is approved.
POST /api/auth/login	Authenticate a user and start a session.
POST /api/auth/logout	End the current session.
GET /api/questions	Return questions permitted for the signed-in role; supports filters.
POST /api/questions	Intern creates a question.
GET /api/questions/:id	Authorized intern/mentor reads question detail.
PATCH /api/questions/:id/status	Authorized mentor updates status.
POST /api/questions/:id/responses	Mentor adds a response.
POST /api/questions/:id/comments	Authorized participant adds a follow-up comment.
GET /api/dashboard/summary	Return role-scoped dashboard counts.

All endpoint names are proposed and may change after supervisor review. Protected endpoints must enforce authorization server-side.

08 | Test Plan & Traceability

Test ID / requirement	Steps	Expected result
T-01 / FR-01	Submit valid login credentials.	User reaches the correct role dashboard.
T-02 / FR-02	Submit a question with all required fields.	Exactly one question is created and shown in My Questions.
T-03 / FR-02	Submit a question with missing required fields.	Validation errors appear; no question is created.
T-04 / FR-03, FR-10	Intern A tries to access Intern B's question ID.	Server denies access or returns not found.
T-05 / FR-05	Mentor adds a response to a valid question.	Response is saved with author and timestamp.
T-06 / FR-06	Unauthorized intern calls status-update endpoint.	Server denies the operation.
T-07 / FR-07	Authorized user adds a follow-up comment.	Comment appears in the correct question thread.
T-08 / FR-08	Filter questions by status.	Only authorized matching questions appear.
T-09 / NFR-03	Use keyboard navigation and a mobile-width viewport.	Primary form remains usable and fields have clear labels.
T-10 / NFR-02	Measure list response with seeded demo data.	Response time is compared with the proposed 2-second target.

Test Evidence to Record Later

For each test, record test ID, environment, precondition, steps, expected result, actual result, pass/fail, and any issue link. This planning document defines test cases; it does not claim the tests have been executed.

Definition of Done — Proposed

Acceptance criteria for the feature are met.
Input validation and role/ownership checks are in place.
Related test cases pass and failures are documented.
Documentation reflects the implemented behavior.
No secrets or real private data are committed to the repository.

09 | Execution Plan, Risks & Handover

Prioritized Backlog

ID	Task	Priority	Deliverable
T01	Create repository, README and proposed folder structure	High	Repository and setup documentation
T02	Confirm requirements, status lifecycle and technology stack	High	Supervisor-approved decisions
T03	Set up React frontend project	High	Frontend shell running locally
T04	Set up Node.js/Express API	High	Basic API server
T05	Create PostgreSQL schema and migrations	High	Initial database schema
T06	Implement authentication and role-based access	High	Protected intern/mentor access
T07	Implement question submission and validation	High	Submit Question workflow
T08	Implement question list and detail views	High	My Questions and question details
T09	Implement mentor responses and status updates	High	Mentor workflow

T10	Implement comments and search/filter	Medium	Follow-up and filtering
T11	Implement dashboard summaries	Medium	Role-specific dashboard
T12	Run tests, fix issues and update documentation	High	Test evidence and updated README

Risks & Mitigations

ID	Risk	Impact	Mitigation
R01	Supervisor approval of the stack is delayed	High	Confirm proposed stack early and document alternatives if requested.
R02	Requirements change during planning	Medium	Keep scope and decision log; assess changes against MVP.
R03	Database design misses a required relationship	High	Review ERD against requirements and user journeys.
R04	Authorization or ownership checks are missed	High	Make server-side permission checks a core requirement and test case.
R05	Time is limited before the deadline	High	Prioritize the primary workflow and follow milestones.
R06	PRD and diagrams use inconsistent terminology	Medium	Cross-check statuses, roles, field names and architecture across deliverables.
R07	Repository contains secrets or private data	High	Use synthetic data, environment variables and .gitignore.